

RNNs, LSTMs, GRUs, and Transformers

Course: CSE 711 (Symbolic Machine Learning-I)
Semester: Spring 2026

Presented By:

1. S M Tasnimul Hasan (ID: 1000060032)
2. Noholi Islam Duti (ID: 1000058446)
3. Miftaun Noor (ID: 21241021)
4. Md. Ashfaqur Rahman (ID: 1000060031)



Inspiring Excellence

01

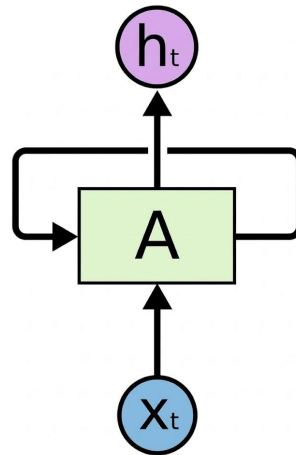
Recurrent Neural Network (RNN)

What is RNN?

- A type of neural network designed to process **sequential data**.
- It has **memory**
- It processes input **step by step**.
- Real-world problems always depend on **previous context**:
 - “I love machine ____” → needs to remember “machine” to guess “learning”
 - Speech recognition (“Hey Google, play...”)
 - Stock prices, weather forecasts, machine translation

Key Idea:

“RNNs remember the past to improve the future.”



RNN Architecture

Core Components:

- ❑ **Input (x_t):** Current data at time step t .
- ❑ **Hidden State (h_{t-1}):** The "memory" from the previous step.
- ❑ **Activation Functions:** The **tanh** (creates new memory) and **softmax** (makes predictions).
- ❑ **Output (y_t)**

The Workflow (Memory Update):

- **Transform:** Multiply previous memory (h_{t-1}) and current input (x_t) by their weights.
- **Combine:** These transformed inputs are added together, along with a bias term (b).
- **Update:** Pass through **tanh** activation function to create the new hidden state (h_t).
- **Predict:** Branch off (h_t) processed through the **Softmax** to get the final output (y_t).

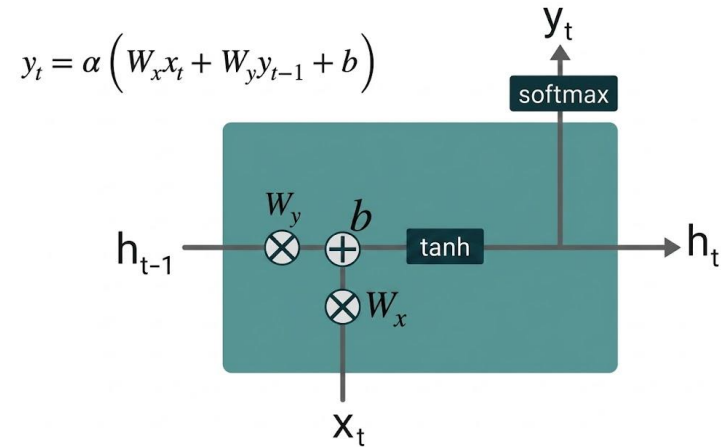


Figure: RNN Architecture

Strengths and Limitations of RNN

Advantages:

- Handles **sequential & variable length data**
- Maintains historical **context** (memory) for accurate predictions.

Limitations:

- **Slow Training:** Must process data sequentially step-by-step; cannot easily parallelize like other models.
- **Long-Term Dependencies:** Struggles to remember information from early inputs in long sequence.

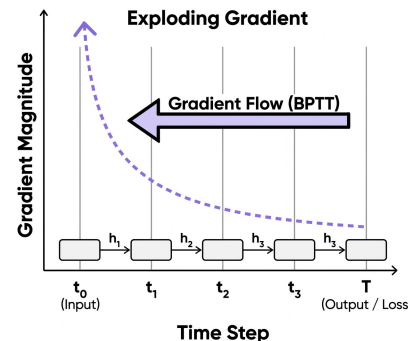
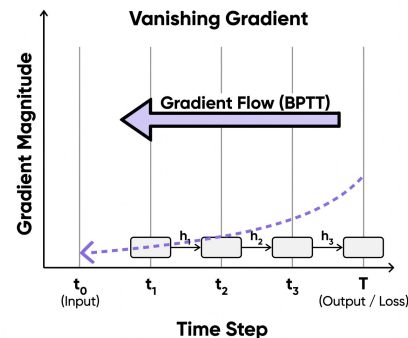
The Gradient Problem & Solutions

The Core Issues:

- **Vanishing Gradients:** Update signals shrink exponentially toward zero. Early layers stop learning.
- **Exploding Gradients:** Update signals grow infinitely large. The model becomes wildly unstable or crashes.

The Solutions:

- **Simplify:** Reduce the number of hidden layers within the neural network, which will reduce the complexity in RNNs.
- **Evolve:** Use advanced RNN architectures such as **LSTM** and **GRU**.



Introduction to Long Short-Term Memory (LSTMs)

LSTM is a special type of RNN designed to handle long-term dependencies in sequential data.

- Introduced to overcome RNN limitations
- Improved version of RNN
- Designed to capture long-term dependencies

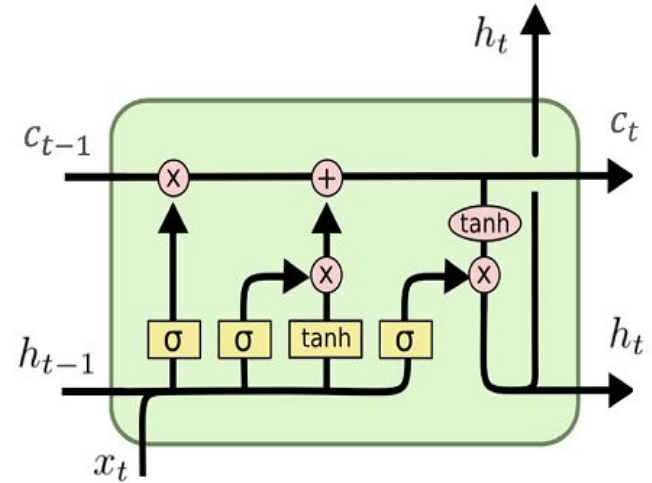
Example:

I love machine learning, so I want to build a career in _

- RNN may forget earlier information
- LSTM remembers it and predicts correctly

LSTM Architecture

- ❑ Maintains **long-term** information using a **Cell State**
- ❑ Produces a **Hidden State** as **short-term** output
- ❑ Uses gates to control information flow
- ❑ Gates apply activation functions like sigmoid and tanh
- ❑ Allows selective addition and removal of information from memory



LSTM
(Long-Short Term Memory)

LSTM Gates

Forget Gate:

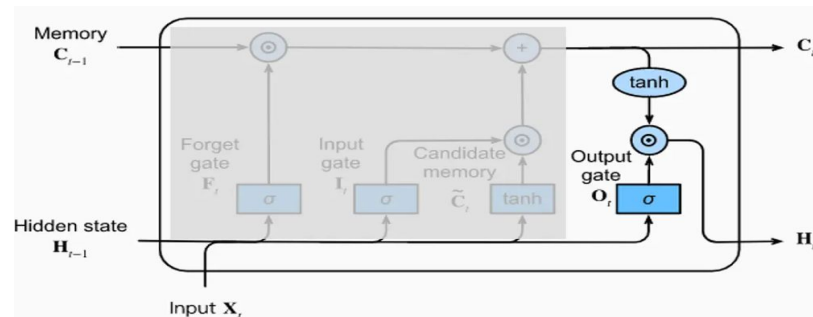
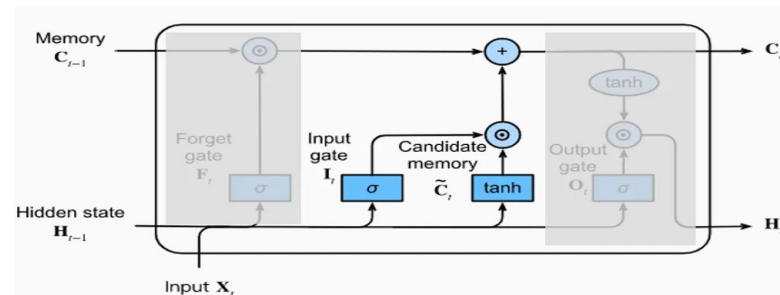
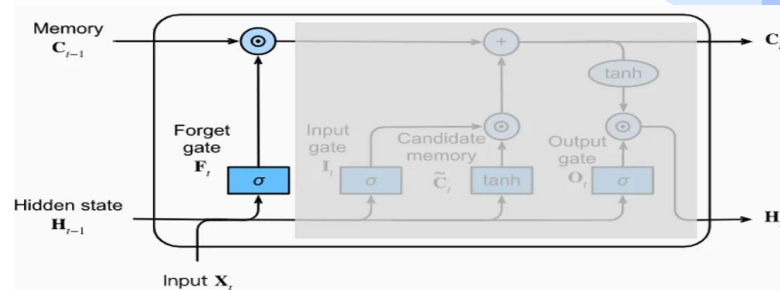
- Decides what information to discard
- $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$

Input Gate:

- Selects what new information to store
 - $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$
- Creates candidate values
 - $\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$
- Cell State Update: Combines old memory and new information
 - $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$

Output Gate:

- Controls what information to output from the cell state
 - $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$
- Produces next hidden state
 - $h_t = o_t * \tanh(C_t)$



LSTM: Advantages and Limitations

Advantages

- Reduces vanishing gradient problem
- Captures long-term dependencies
- Performs selective memory filtering
- Robust for sequential data

Limitations

- Computationally expensive
- Slower training
- Risk of overfitting
- Complex architecture

Due to these limitations, a simpler model called **GRU** was introduced.

Introduction to Gated Recurrent Units (GRUs)

GRU is a gated RNN that controls information flow using **update** and **reset** gates to model long-term dependencies efficiently.

- Addresses RNN memory limitations (vanishing gradient)
- Simpler architecture compared to LSTM
- Fewer gates → faster and more efficient

Example: “I grew up in France, so I can speak ____.”

RNN → may forget “France” → wrong prediction

GRU → keeps important context → predicts French

GRU Architecture

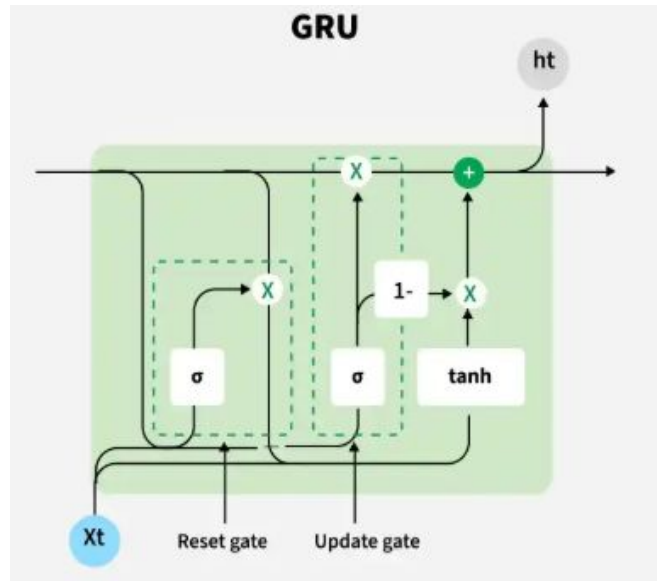
Core Idea: GRU controls information flow using two gates.

Key components:

1. **Update Gate (z)** - decides how much past information to keep
1. **Reset Gate (r)** - decides how much past information to forget

Hidden State Update:

- $$h_t = (1 - z)h_{t-1} + zh_t$$



GRU: Advantages, Disadvantages & Limitations

Advantages:

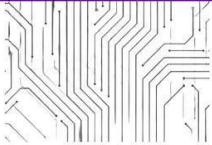
- Simpler & fewer parameters
- Faster training, less memory
- Handles long-term dependencies
- Reduces vanishing gradient

Disadvantages:

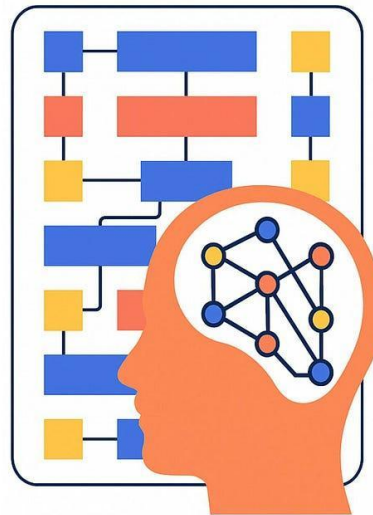
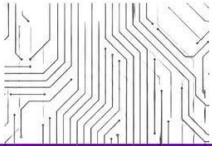
- Less expressive than LSTM (simpler gating)
- May be less effective for very complex or long dependencies
- No separate cell state → less control over memory

Limitations

- Sequential → slower for long sequences
- Needs sufficient data
- Not as powerful as newer models (e.g., Transformer)



Introduction to Transformer Architectures



Transformer

A **Transformer** is a type of deep learning model designed to understand and process sequences of data especially text using a mechanism called **attention**.

Instead of reading data **word-by-word** like older models (RNNs, LSTMs, GRUs), a Transformer looks at **all words of the input at the same time** and figures out which words are important for understanding each other.

Intuitive Example:

“The dog chased the ball because it was fast.”

- The model needs to understand what **“it”** refers to.
A Transformer compares all words at once and learns that **“it”** refers to **the dog**, not the **ball**.
- The **Transformer model** was first introduced in **2017**. It came from a research paper titled: **“Attention Is All You Need”**
- The paper was written by researchers at **Google (Google Brain team)**.

Transformer Architecture

Example:

Input: "I love AI"

Output: "আমি AI ভালোবাসি"

1. Input Embedding

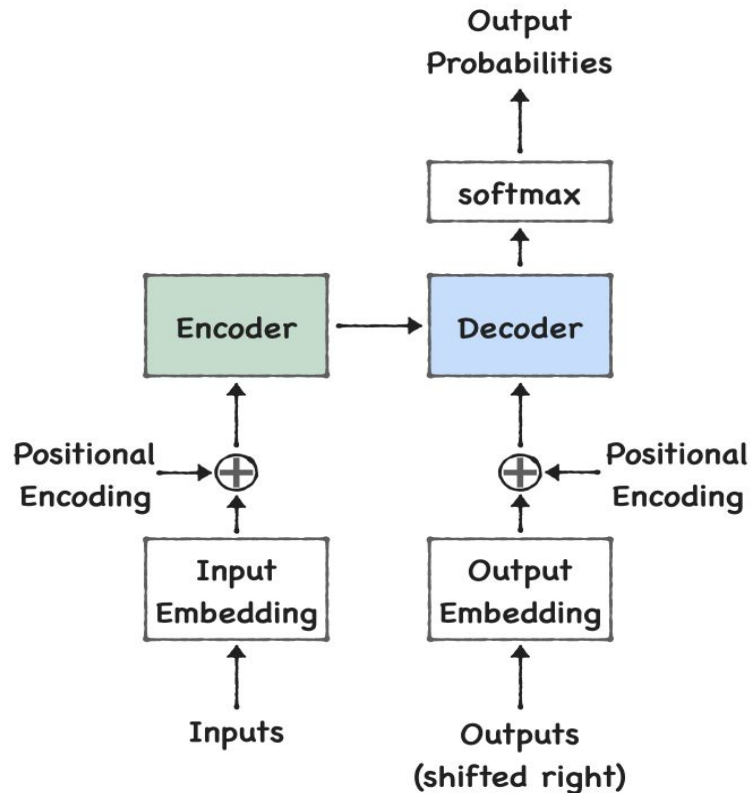
- Convert words $\rightarrow$ vectors
"I", "love", "AI" $\rightarrow$ numerical form

2. Positional Encoding

- Add word order information
Helps model understand sequence

3. Encoder (Understanding Input)

- Uses **self-attention**
- Learns relationships:
"I" $\rightarrow$ subject, "love" $\rightarrow$ action,
"AI" $\rightarrow$ object



Transformer Architecture

4. Decoder Input (Shifted Right)

- Starts with: `<start>` আমি ...
- Generates output step by step

5. Decoder (Generating Output)

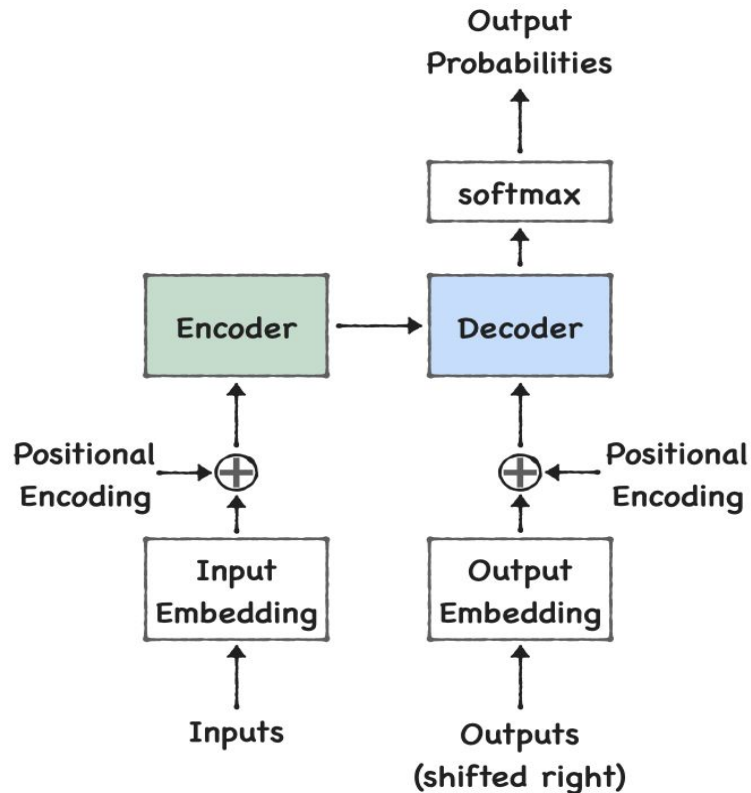
- Uses:
 - Previous words
 - Encoder output (input meaning)
Maps: “I” → “আমি”, “love” → “ভালোবাসি”

6. Softmax Layer

- Predicts next word using probability
Chooses most likely word

Final Output (Word-by-Word)

“আমি” → “AI” → “ভালোবাসি”



Transformer Model

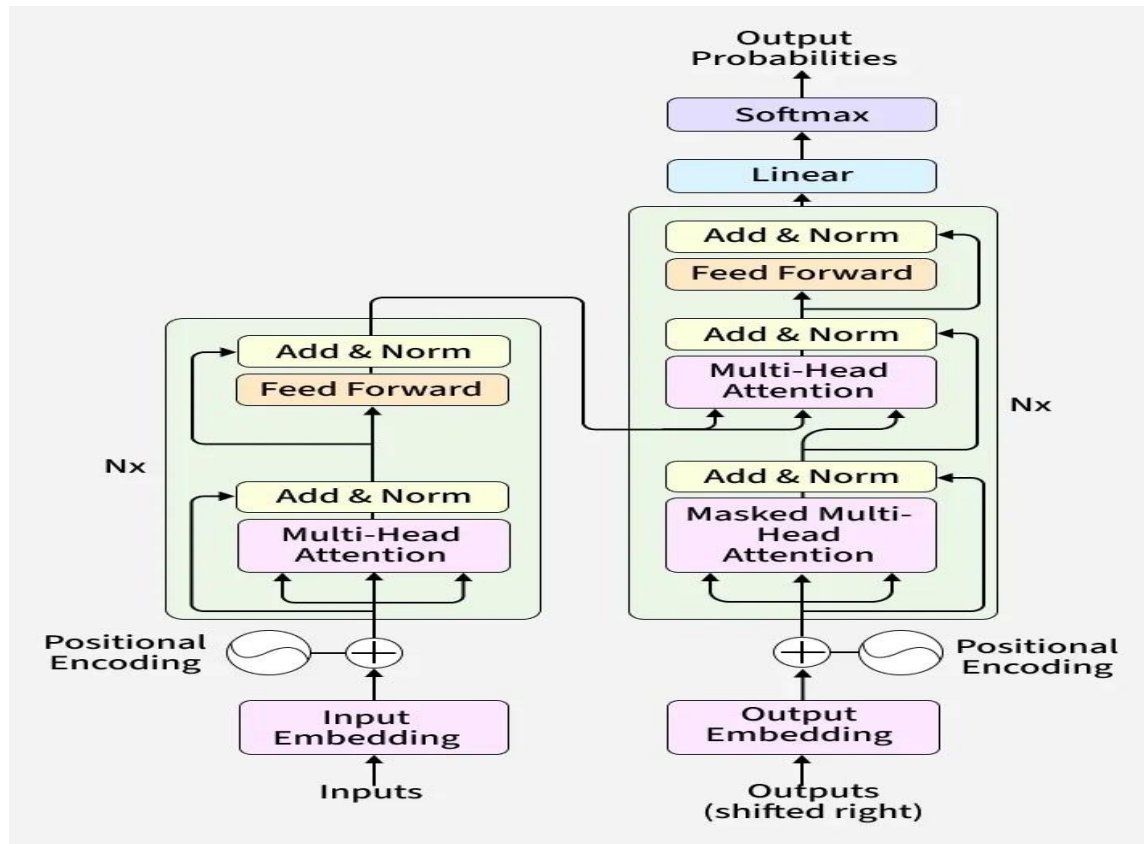


Figure: Transformer Model (from GeeksforGeeks)

Transformer

Common Transformer Architectures

- **Encoder-only (BERT, RoBERTa):** Best for text classification, named entity recognition, and sentiment analysis.
- **Decoder-only (GPT, LLaMA):** Best for generative tasks (e.g., text generation, chat).
- **Encoder-Decoder (T5, BART):** Best for sequence-to-sequence tasks like translation and summarization

Transformer

Advantages

Parallel Processing – Processes all words at once, making training much faster

Better Context Understanding – Captures long-range dependencies effectively

No Vanishing Gradient Problem – Unlike RNNs/LSTMs

Scalable Architecture – Easy to build larger and more powerful models

Challenges

High Computational Cost – Requires powerful GPUs/TPUs

Large Memory Usage – Attention mechanism uses a lot of memory

Needs Large Data – Performs best with huge datasets

Complex Architecture – Harder to understand and implement

Applications of Transformer Architecture



Natural Language Processing (NLP):

Powering models like BERT, GPT, and T5.



Machine Translation:

Used in Google Translate and similar applications



Text Summarization:

Enabling AI-driven summarization tools



Speech Recognition:

Enhancing voice assistants like Alexa and Siri



Computer Vision:

Applied in vision transformers (ViTs) for image processing

Conclusions

Overall, **Transformer** outperformed RNN, LSTM, and GRU by using attention and parallel processing, making them faster, more scalable, and better at understanding context.



Thanks!

Do you have any questions?

